

PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

MAGÍSTER EN INTELIGENCIA ARTIFICIAL

INF3862 PLATAFORMAS BIG DATA PARA DATA SCIENCE

Recetas acordes al gasto energético

Proyecto del curso, Entrega 1: Diseño de datos

Alumno: Julynho Merlin Toledo Ríos

Profesor: Sebastián Hagedorn

Ayudante: Tomás Werth

Noviembre de 2026

Entrega 1: Diseño de datos

1. Objetivo del proyecto

La app ya sabe qué recetas prepara cada persona y, desde que se conectaron los relojes, también sabe cuánto se mueve. Lo que quiero hacer en este proyecto es juntar esas dos cosas: que la app le proponga a cada usuario, todos los días, recetas cuyo aporte de calorías y proteína vaya de acuerdo con la energía que está gastando. La idea es simple, alguien que viene de una semana con mucha actividad debería recibir sugerencias distintas a las de alguien que pasó varios días en reposo. Para llegar a esa recomendación primero necesito estimar cuánto va a gastar cada persona al día siguiente, así que esa predicción es el paso intermedio del proyecto. De pasada, el equipo de producto se queda con un panel que compara lo que los usuarios gastan con lo que registran como comida, y que les sirve para decidir qué recetas destacar. En la Tabla A1 respondo las cinco preguntas de la Clase 0 con las cifras que saqué al revisar los archivos.

Al revisar los datos hubo dos cosas que terminaron marcando casi todo el diseño. La primera es el tamaño: las mediciones del reloj son prácticamente todo el volumen (15,5 millones de filas en 61 días para solo 300 usuarios, 2,24 GB en JSON) y, al lado de eso, las demás fuentes son pequeñas. La segunda es el tiempo. Me di cuenta de que la mitad de las mediciones llega a la app más de 80 minutos después de haber sido tomada, y una de cada ocho llega con más de seis horas de atraso, porque el reloj solo envía cuando sincroniza con el teléfono. Por eso decidí que la recomendación se calcule una vez al día, ya que no tiene sentido exigir tiempo real cuando la misma fuente no entrega los datos a tiempo.

2. Fuentes y recepción de datos

Trabajo con las cuatro fuentes del wearable y con el catálogo de recetas. Las reseñas las dejé fuera por ahora, porque el catálogo ya trae la calificación promedio de cada receta y con eso me alcanza para filtrar lo que se sugiere. El detalle de cada fuente está en la Tabla B1, pero lo que más me sirvió fue entender que los datos llegan de tres formas distintas. Las comidas y la vinculación de relojes se registran dentro de la misma app, una por una, así que ahí conviene validar el dato en el momento en que se escribe. Las mediciones y las sesiones, en cambio, llegan en lotes cada vez que el reloj sincroniza, unas diez veces al día, y es en esta fuente donde encontré los problemas. Los dos modelos de reloj nombran distinto los mismos campos (uno incluso manda la hora como un número en milisegundos) y el 2,2 % de las mediciones viene repetido porque el reloj reintenta el envío. Además hay atrasos de hasta 4,5 días, y la frecuencia cardíaca viene vacía en el 2,0 % o el 4,3 % de los registros, según el modelo. Para estos datos prefiero guardar cada lote tal como llegó y ordenar los nombres y los tipos después, al momento de leerlos. Lo hago así porque, si más adelante aparece un tercer modelo de reloj o descubro que una de mis reglas de limpieza estaba mal, puedo volver a procesar desde el original. El catálogo es el caso más simple, porque llega como un archivo completo que se carga cada cierto tiempo.

También hice la cuenta de cuánto crecería esto. Hoy entran unas 255 mil mediciones al día, que son 37 MB en JSON y menos de 2 MB comprimidos, o sea, muy poco. Pero si los 271.907 usuarios que han escrito reseñas conectaran un reloj, serían cerca de 230 millones de mediciones y 33 GB diarios. Traté de que el diseño aguante ese salto cambiando solo el motor con que se procesa, sin tener que reorganizar los datos.

3. Estructura operacional (OLTP)

Para el sistema que hace funcionar la app (Tabla C1 y Figura C1) me quedé principalmente con el modelo relacional. Usuarios, relojes, recetas, reseñas, comidas y sesiones se referencian entre sí por sus claves, se leen y se escriben de a pocas filas y necesitan reglas, que es justo el caso en que la Clase 1 recomienda partir por ese modelo. Hice tres excepciones pensando en cómo se usa cada dato. Las mediciones van como series temporales, porque solo se agregan, siempre se consultan por reloj y por rango de tiempo, y son muchísimas más que todo lo demás; además, al usar como clave el reloj junto con el instante de la medición, si un lote llega dos veces no se duplica nada. El resumen del día y la recomendación vigente de

cada usuario van en un almacén clave-valor, porque la app los pide por identificador a cada rato. Y las imágenes de las recetas van en almacenamiento de objetos, dejando en la receta solo la dirección.

La regla de integridad que elegí para mostrar una transacción es que un usuario tenga un solo reloj activo a la vez. Cambiar de reloj son dos escrituras, cerrar el vínculo anterior y abrir el nuevo, y las dos tienen que confirmarse juntas. Si solo se hace la primera, las mediciones que lleguen después no tienen dueño; si solo se hace la segunda, la persona queda con dos relojes activos y su gasto del día se podría contar dos veces. Y esto pasa de verdad en los datos. Cuando revisé a los 15 usuarios que cambiaron de reloj, en 6 casos los dos vínculos aparecen activos al mismo tiempo durante algunas horas (hasta 11,9), y en los otros 9 la persona pasó un rato sin ningún reloj vinculado.

4. Estructura analítica (OLAP)

Para el análisis propongo dos tablas de hechos que comparten dimensiones (Tablas D1 y D2). La principal es `hechos_actividad_diaria` y tiene una fila por reloj y día local del usuario, con las calorías gastadas, los pasos, la frecuencia cardíaca media y máxima, los minutos que realmente se midieron y un resumen de las sesiones. Elegí el día como grano porque mi objetivo se piensa en días y no en minutos, y porque así 15,2 millones de mediciones únicas quedan en 16.330 filas. Un detalle que casi se me pasa es que el día tiene que ser el de la hora local de cada persona, porque si cortara por UTC la tarde y la noche de alguien que vive en Los Ángeles quedarían repartidas en dos días. La segunda tabla, `hechos_comida`, tiene una fila por cada comida registrada, con sus porciones y el aporte nutricional de la receta. Las dimensiones son fecha, usuario, reloj y receta. De todos modos las mediciones limpias también se guardan completas, porque las voy a necesitar para las variables de ventana de la E3 y para recalcular cuando lleguen datos atrasados.

Sobre dónde guardar todo esto, me decidí por un Lakehouse. Los datos crudos quedan en almacenamiento de objetos tal como llegan, y los limpios y analíticos quedan como tablas en Parquet manejadas con un formato de tabla como Delta o Iceberg (Tabla D3). Pensé en un Warehouse, pero lo descarté porque necesito conservar los lotes originales, con sus dos esquemas, para poder reprocesarlos, y porque el modelo de la E3 va a leer los archivos directamente. También pensé en un Lake simple, solo con archivos, pero los atrasos me obligan a corregir días que ya estaban cargados, y ahí necesito que la corrección se publique completa o no se publique, además de poder volver después a la versión exacta con la que entrené. El almacenamiento es por columnas, porque las consultas de análisis usan tres o cuatro columnas de millones de filas y los valores que se repiten mucho, como el reloj, la fecha o los pasos, se comprimen bien. Por último, las mediciones limpias las particiono por la fecha en que se midieron y no por la fecha en que llegaron. Lo decidí al ver que el 14,4 % de las mediciones viene en la carpeta de un día distinto al que fue tomada, y que todo lo que quiero preguntar tiene que ver con cuándo ocurrió la actividad. Con esa partición, las consultas por rango de fechas y el reproceso de los últimos días abren solo las carpetas que necesitan. Lo que no mejora es buscar a un solo usuario en todo el historial, y eso lo dejo anotado.

5. Consultas

En el Anexo E dejé siete consultas con su pseudocódigo y su justificación. Las tres transaccionales tienen en común que tocan pocas filas, se ubican por clave y tienen que responder al instante aunque haya muchos usuarios a la vez: el cambio de reloj (T1, donde muestro el `BEGIN` y el `COMMIT`), la recepción de un lote de mediciones (T2, que escribe unas 60 filas y se puede repetir sin duplicar) y la lectura de la recomendación del día (T3). Las tres analíticas hacen lo contrario, recorren muchas filas para devolver pocas, y pueden demorarse segundos o minutos sin problema. A1 es la que responde directamente el objetivo, porque toma el gasto promedio reciente de cada usuario y lo cruza con el catálogo para proponerle recetas en su rango. A2 arma el balance diario entre lo gastado y lo registrado como comida, y A3 mide qué parte de las mediciones de cada día ya había llegado a cierta hora, que es justo lo que necesito para decidir a qué hora correr el proceso diario.

La que me costó clasificar fue el resumen de hoy que ve el usuario cuando abre la app (F1). Por cómo está escrita parece analítica, porque suma hasta unas dos mil mediciones, pero por cómo se usa es operacional: la pide una persona, sobre un solo reloj, varias veces al día y esperando verla al instante. Al final la traté como carga operacional. No la correría en el Lakehouse, que va con horas de desfase, ni tampoco haría la suma sobre la serie cada vez que alguien abre la app. Lo que propongo es recalcular el resumen cuando llega cada lote y que la app solo lo lea por clave. Distinto sería si ese mismo indicador se pidiera para todos los usuarios juntos, por ejemplo en un ranking del día, porque ahí ya es una consulta analítica y la mandaría a una réplica de lectura o al Lakehouse, aceptando el desfase.

6. Arquitectura inicial

La Figura 1 muestra el recorrido completo. Hay tres caminos de entrada, uno por cada forma en que llegan los datos, y el servicio de sincronización escribe cada lote en dos lugares: en la serie temporal, para que la app pueda mostrar la actividad reciente, y sin tocar en los datos crudos, que son el punto de partida del análisis. Separé la operación del análisis para que una consulta larga no le quite recursos a la app. Los dos lados se conectan solo en dos puntos, la extracción diaria de las tablas operacionales y la publicación de la recomendación de vuelta al almacén clave-valor, y así la app nunca consulta directamente las tablas analíticas. Preferí dejar el diseño pequeño, porque para el volumen de hoy basta con un proceso batch diario sobre archivos. Lo que podría cambiar con la escala lo dejé anotado en el registro de decisiones para revisarlo en la E2.

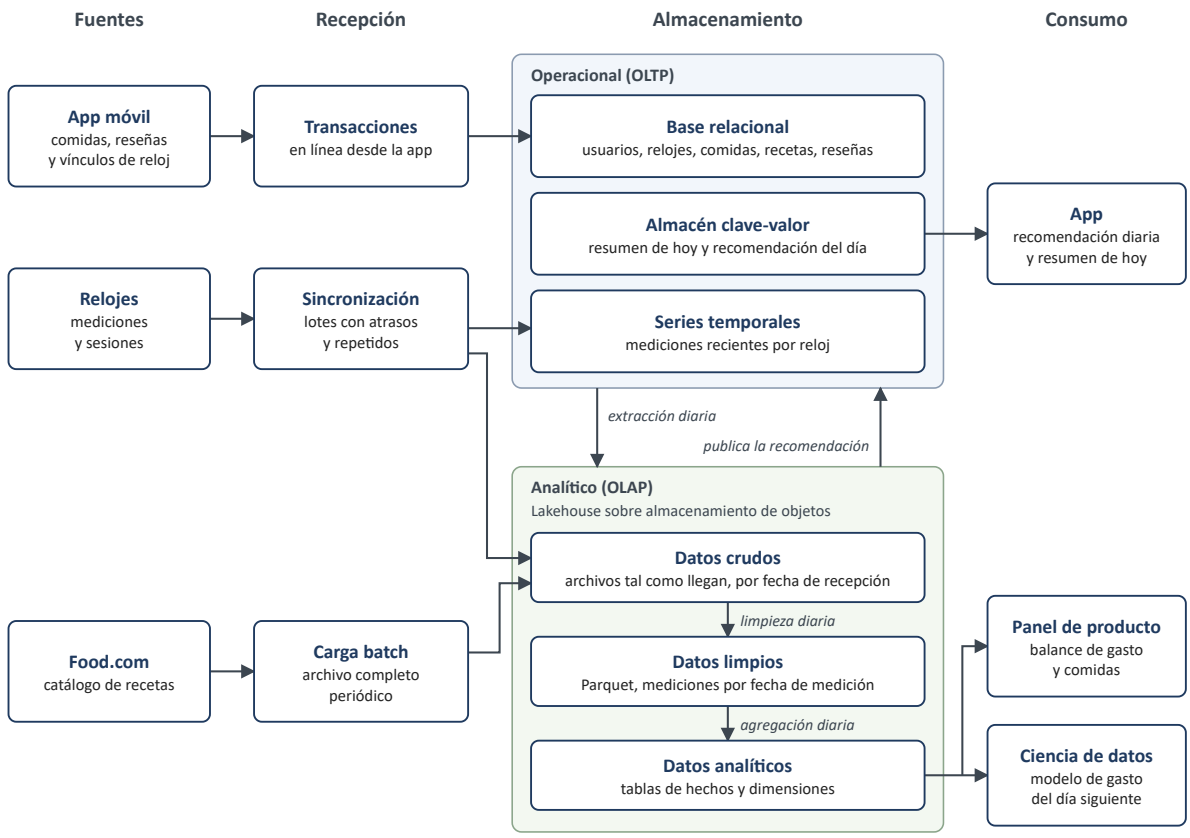


Figura 1. Arquitectura inicial por capacidades: fuentes, recepción, almacenamiento operacional y analítico, y consumo. Las flechas indican el sentido en que viajan los datos.

Anexos

Todas las cifras las saqué revisando los archivos entregados, que cubren del 16 de diciembre de 2007 al 14 de febrero de 2008. Los nombres de tablas y columnas van sin tildes porque son identificadores.

Anexo A. Objetivo y preguntas guía

Tabla A1. Las cinco preguntas de la Clase 0 aplicadas al caso de uso.

Pregunta	Respuesta
1. Resultado	Una recomendación diaria de recetas dentro de la app, para el usuario con reloj vinculado, cuyo aporte de calorías y proteína es acorde con su gasto energético. Apoya la decisión de qué cocinar hoy. Se sostiene en una predicción del gasto calórico del día siguiente por usuario. Producto secundario: un panel para el equipo de producto con el balance entre gasto y comidas registradas, que apoya la decisión de qué recetas destacar.
2. Datos	Mediciones del reloj: 15.540.132 filas en 1.449 archivos (115,5 MB comprimidos, 2,24 GB en JSON), que llegan en lotes al sincronizar. Sesiones de actividad: 4.840. Relojes vinculados: 315, de 300 usuarios. Comidas registradas en la app: 22.106. Catálogo de recetas: 522.517 filas (178,7 MB en Parquet), recibido como archivo completo.
3. Operaciones	Consultar el historial de un usuario y la ficha de una receta. Transformar: unificar los dos esquemas de reloj, eliminar mediciones repetidas, pasar a hora local, agregar por reloj y día, y cruzar las comidas con el catálogo. Predecir el gasto calórico del día siguiente y, con él, filtrar el catálogo por rango de calorías y proteína.
4. Exigencias	La recomendación se calcula una vez al día y tolera horas de espera; a las 06:00 UTC del día siguiente ya llegó el 94,6 % de las mediciones de un día. La app, en cambio, debe responder en milisegundos. Es tolerable un error de algunos cientos de kilocalorías en el gasto estimado, porque el resultado es una sugerencia. No es tolerable contar dos veces una medición, perder la relación entre reloj y usuario ni exponer datos de actividad y frecuencia cardíaca de una persona.
5. Recursos	Me imagino un equipo de datos chico, de dos o tres personas que manejan SQL y Python, con un computador para desarrollar y probar, y servicios administrados de bajo costo para operar, sin un clúster propio. El presupuesto mensual es acotado y lo voy a estimar en la E3.

Supuestos que tomé.

1. Revisé la documentación que viene con los datos en Canvas y no encontré si las calorías y los demás nutrientes del catálogo son por porción o por la receta completa. Decidí tomarlos como valores por porción, que es como Food.com los muestra en su sitio.
2. Tampoco queda claro si el campo `porciones` de las comidas corresponde a lo que se preparó o a lo que comió el usuario. Asumí que son las porciones preparadas, así que a cada comida le atribuyo el aporte de una sola porción. Además, las comidas registradas (1,2 por usuario y día) no son todo lo que la persona come, y por eso leo el balance como una referencia.
3. La parte del gasto diario que le asigno a una comida la dejé como un parámetro que el negocio puede ajustar. No pretende ser una indicación nutricional.

Anexo B. Fuentes y recepción

Tabla B1. Fuentes que usa el caso de uso.

Aspecto	Mediciones	Sesiones	Relojes	Comidas	Recetas (Food.com)
Un registro es	Una lectura del reloj en un instante: frecuencia cardíaca, pasos y calorías desde la lectura anterior.	Una actividad registrada (caminar, correr, bicicleta o fuerza) con inicio, fin, distancia y calorías.	Un reloj vinculado a un usuario, con modelo, fechas de vinculación y desvinculación y zona horaria.	Una receta preparada por un usuario en una fecha y hora, con sus porciones.	Una receta del catálogo con autor, tiempos, categoría, ingredientes, instrucciones y nutrientes.
Tipo	Semiestructurado: JSON por línea, con dos variantes de campos.	Semiestructurado: JSON por línea.	Estructurado.	Estructurado.	Estructurado, con columnas de lista y texto libre.
Volumen entregado	15.540.132 filas; 115,5 MB comprimidos y 2,24 GB sin comprimir.	4.840 filas (4.792 únicas); 0,8 MB.	315 filas; 20 KB.	22.106 filas; 1,0 MB.	522.517 filas; 178,7 MB en Parquet.
Crecimiento estimado	255 mil filas y 37 MB de JSON al día con 300 usuarios (7,6 millones y 1,1 GB al mes). Con 271.907 usuarios, unos 230 millones de filas y 33 GB al día.	79 al día (2.400 al mes) con 300 usuarios; unas 71 mil al día con 271.907.	Crece con los usuarios y no con el uso: una fila por reloj vinculado.	362 al día (11 mil al mes) con 300 usuarios; unas 328 mil al día con 271.907.	Unas 25 mil recetas nuevas al año, según las fechas de publicación entre 1999 y 2020.
Modo de recepción	Eventos en lotes: cada sincronización entre reloj y teléfono envía un grupo de lecturas.	Evento al terminar la actividad, que viaja en la misma sincronización.	Transacción en línea desde la app. Al análisis llega por extracción completa diaria.	Transacción en línea desde la app. Al análisis llega por extracción incremental diaria.	Carga batch de un archivo completo.
Frecuencia y latencia aceptable	Continua, unas 10 sincronizaciones por reloj y día (2.567 lotes diarios). Latencia aceptable de horas, con reproceso de los últimos 5 días.	Unas 0,26 por usuario y día. Latencia de horas.	Ocasional. Inmediata en la app y de un día para el análisis.	1,2 por usuario y día, concentradas en tres franjas horarias. Un día para el análisis.	Semanal o mensual. Latencia de días.
Problemas esperados	Dos esquemas según el modelo; 2,2 % de filas repetidas; atraso mediano de 80 minutos y máximo de 4,5 días; frecuencia cardíaca vacía en 2,0 % y 4,3 %; 13 lecturas fuera de 30 a 220 latidos.	48 sesiones repetidas. Distancia vacía en fuerza, que es un valor válido. Sus calorías ya están contenidas en las mediciones.	Cambios de reloj con vínculos solapados (6 de 15) o con horas sin reloj (9 de 15).	48 comidas con receta inexistente; 1.170 anteriores a la vinculación del reloj; registro parcial de lo que se come.	Identificador guardado como decimal; calorías en 0 en 3.111 recetas y sobre 5.000 en 2.615; porciones vacías en 182.911; tiempos como texto ISO 8601.
Esquema	Al leer. Se conserva el lote original y se unifica en la capa limpia.	Al leer, por la misma razón.	Al escribir, con claves y restricciones.	Al escribir, validando usuario y receta.	Al escribir en el catálogo de la app; al leer para las listas del archivo.

Anexo C. Modelo operacional (OLTP)

Tabla C1. Entidades del sistema operacional, claves y modelo de datos elegido.

Entidad	Clave primaria	Claves foráneas	Columnas principales	Modelo y motivo
usuario	author_id	No tiene	nombre, correo, zona_horaria, fecha_registro	**Relacional.** Casi todas las demás tablas la referencian y se accede a ella por clave.
dispositivo	device_id	author_id	modelo, fecha_vinculacion, fecha_desvinculacion	**Relacional.** Sostiene la regla de un solo reloj activo por usuario, que necesita restricciones y transacciones. Si un reloj pudiera pasar a otra persona, el vínculo se separaría en una tabla propia.
receta	recipe_id	autor_id	nombre, categoría, tiempos, nutrientes, porciones; listas de ingredientes, instrucciones y palabras clave	**Relacional con columnas de lista.** Comidas y reseñas la referencian por clave. Las listas se leen siempre junto con la receta, así que no se separan en tablas. Las imágenes van a almacenamiento de **objetos**.
resena	review_id	author_id, recipe_id	calificacion (0 a 5), texto, fecha_envio, fecha_modificacion	**Relacional.** El par usuario y receta es único, y los datos lo confirman: no hay pares repetidos entre 1.401.982 reseñas.
comida	comida_id	author_id, recipe_id	fecha_hora, porciones	**Relacional.** Inserciones de una fila que deben validarse contra usuario y receta.
sesion_actividad	session_id	device_id	tipo, inicio, fin, distancia_km, calorias	**Relacional.** Pocas filas, consultadas por reloj y fecha. La clave descarta los reenvíos.
medicion	device_id + ts_medicion	device_id	ts_recepcion, frecuencia_cardiaca, pasos, calorias	**Series temporales.** Solo inserciones, consulta por reloj y rango de tiempo, 255 mil filas diarias. Se conserva una ventana reciente y el historial vive en el Lakehouse.
resumen_diario y recomendacion_diaria	author_id + fecha	No tienen	pasos, calorías y última medición del día; lista ordenada de recetas sugeridas	**Clave-valor.** Lecturas repetidas por identificador desde la app. Se pueden reconstruir a partir de las otras tablas.

Reglas de integridad. (1) Un usuario tiene como máximo un reloj activo, lo que se garantiza con un índice único sobre `author_id` para las filas sin fecha de desvinculación y con la transacción de la consulta T1. (2) Una reseña por usuario y receta. (3) Una medición por reloj e instante, que vuelve idempotente la recepción de lotes. (4) Una comida exige que existan el usuario y la receta.

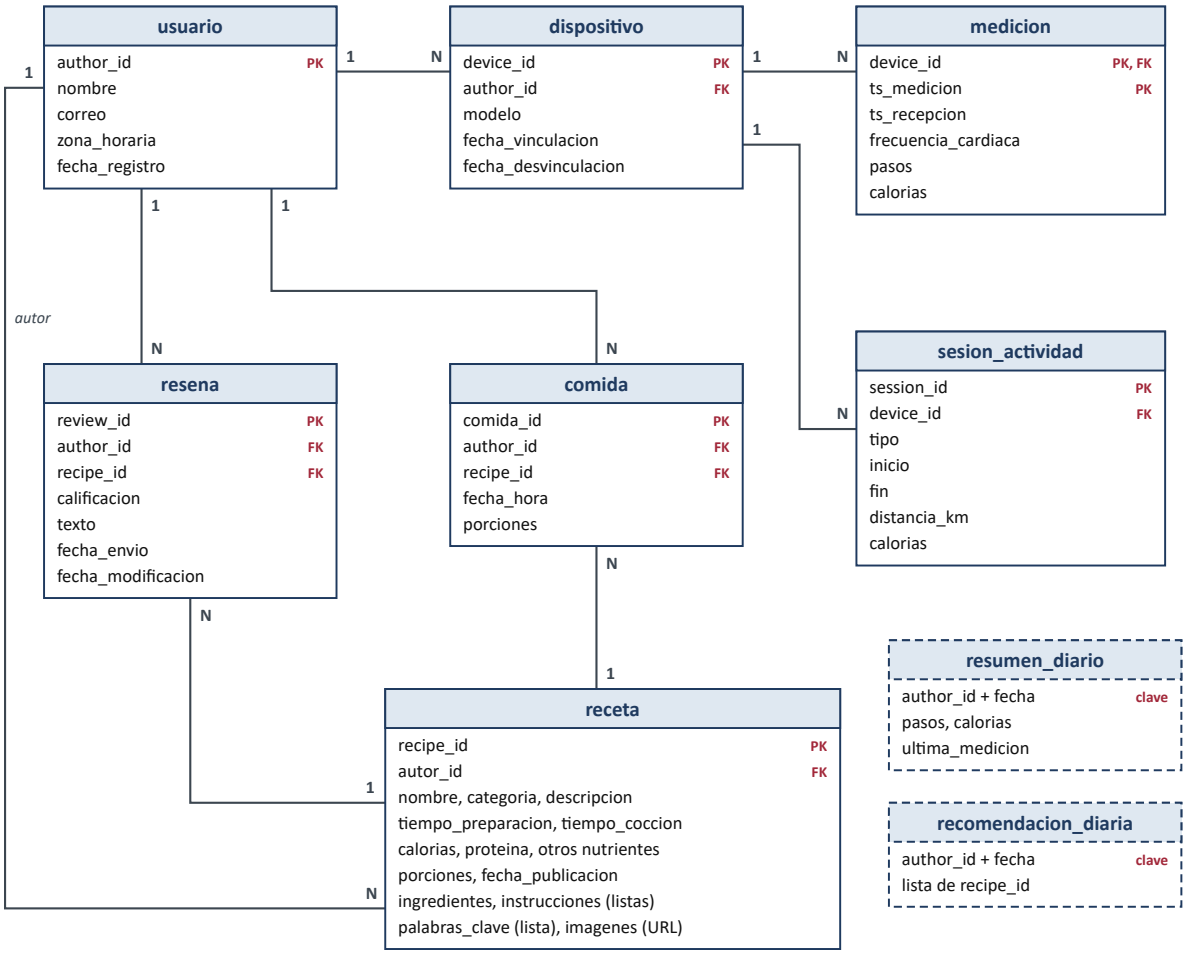


Figura C1. Diagrama entidad-relación del sistema operacional. PK: clave primaria. FK: clave foránea. Los extremos 1 y N indican relaciones de uno a muchos. Las estructuras clave-valor se muestran con borde punteado porque no forman parte del modelo relacional.

Anexo D. Modelo analítico (OLAP)

Tabla D1. Tablas de hechos.

Tabla	Granularidad	Medidas	Claves a dimensiones
hechos_actividad_diaria	Una fila por reloj y día local del usuario. En los datos actuales coincide con usuario y día (16.330 filas), porque nadie tiene mediciones de dos relojes en un mismo día.	kcal_gastadas, pasos, minutos_medidos, fc_media, fc_maxima, n_sesiones, minutos_sesion y kcal_sesion (informativa, porque ya está incluida en kcal_gastadas).	id_fecha, id_usuario, id_reloj
hechos_comida	Una fila por comida registrada (22.106 en el extracto).	porciones, kcal_porcion, proteina_porcion, carbohidratos_porcion, grasa_porcion.	id_fecha, id_usuario, id_receta. La franja del día (desayuno, almuerzo, cena u otra) va como atributo de la fila.

Tabla D2. Dimensiones.

Dimensión	Clave	Atributos
dim_fecha	id_fecha	fecha, día de la semana, indicador de fin de semana, semana, mes, año.
dim_usuario	id_usuario (author_id)	zona horaria, fecha de la primera vinculación, cantidad de reseñas escritas.
dim_reloj	id_reloj (device_id)	modelo, intervalo de muestreo en reposo (60 o 300 segundos), fecha de vinculación y de desvinculación.
dim_receta	id_receta (recipe_id)	nombre, categoría, calorías, proteína, carbohidratos y grasa por porción, porciones, tiempo total en minutos, calificación promedio, número de reseñas, rango calórico.

Tabla D3. Ubicación, formato y organización física de cada nivel de datos.

Datos	Dónde viven	Formato	Organización	Partición
Crudos	Almacenamiento de objetos, sin modificar.	Los originales: JSON por línea comprimido para mediciones y sesiones; CSV y Parquet para los extractos.	Por filas, salvo el Parquet del catálogo.	Mediciones por día y hora de recepción, tal como llegan. El resto por fecha de extracción.
Limpios	Lakehouse: tablas sobre el mismo almacenamiento.	Parquet con formato de tabla (Delta o Iceberg).	Por columnas.	Mediciones por fecha de medición. Sesiones, comidas, relojes y recetas sin partición, por su tamaño.
Analíticos	Lakehouse.	Parquet con formato de tabla.	Por columnas.	Hechos por mes de la fecha local. Dimensiones sin partición.

Consultas que se benefician de la partición por fecha. Las que filtran un rango de días, como A1 (últimos 7 días), A2 y A3; el reproceso diario, que reescribe solo las particiones de los últimos 5 días, y el cálculo de variables de ventana para el modelo. No se beneficia una consulta que recorre todo el historial de un único usuario; para ese caso ayuda ordenar cada archivo por reloj, de manera que las estadísticas de Parquet permitan saltarse bloques.

Anexo E. Consultas de ejemplo

Son tres transaccionales (T), tres analíticas (A) y una de frontera (F). Dejé el mismo pseudocódigo en `entregas/E1/anexos/consultas.sql` para reutilizarlo en la E2.

T1. Cambio de reloj de un usuario (OLTP, escribe, con transacción)

```
BEGIN;
  -- 1) cierra el vinculo vigente del usuario, si existe
  UPDATE dispositivo
     SET fecha_desvinculacion = :ahora
   WHERE author_id = :usuario
     AND fecha_desvinculacion IS NULL;
  -- 2) abre el vinculo con el reloj nuevo
  INSERT INTO dispositivo (device_id, author_id, modelo, fecha_vinculacion)
  VALUES (:reloj_nuevo, :usuario, :modelo, :ahora);
COMMIT;
-- Si algo falla entre 1) y 2): ROLLBACK, y el vinculo anterior sigue vigente.
-- Regla protegida: indice unico sobre (author_id) donde fecha_desvinculacion IS NULL.
```

Propósito y usuario	Registrar que una persona reemplazó su reloj. La ejecuta la app cuando el usuario vincula un dispositivo nuevo.
Clasificación	OLTP.
Justificación	Toca dos filas, una que se actualiza y una que se inserta. Es poco frecuente (15 cambios en 61 días entre 300 usuarios), pero interactiva, con latencia esperada bajo un segundo. La concurrencia por usuario es baja, aunque el teléfono puede enviar la solicitud dos veces.
Sin transacción	Si el proceso falla después del primer paso, el usuario queda sin reloj y sus mediciones llegan sin dueño. Si los pasos se ejecutan por separado o dos solicitudes se cruzan, queda con dos relojes activos. Con la transacción se confirman ambos cambios o ninguno.
Qué la haría rápida	El índice único parcial sobre `author_id`, que sirve a la vez para ubicar el vínculo vigente y para hacer cumplir la regla.

T2. Recepción de un lote de sincronización (OLTP, escribe, idempotente)

```
BEGIN;
  INSERT INTO medicion
         (device_id, ts_medicion, ts_recepcion, frecuencia_cardiaca, pasos, calorias)
  VALUES (:reloj, :t1, :ahora, :fc1, :pasos1, :kcal1),
         (:reloj, :t2, :ahora, :fc2, :pasos2, :kcal2)   -- ... una fila por lectura del lote
  ON CONFLICT (device_id, ts_medicion) DO NOTHING;      -- reenviar el lote no duplica
COMMIT;                                                 -- solo entonces se confirma al telefono
```

Propósito y usuario	Guardar las lecturas que envía un reloj al sincronizar. La ejecuta el servicio de sincronización.
Clasificación	OLTP.
Justificación	Escribe las filas de un lote, 62 en la mediana y 3.949 en el mayor. Hoy son unos 2.567 lotes diarios. Debe confirmar en menos de un segundo, porque si el teléfono no recibe la confirmación reintenta, y ese reintento es el origen del 2,2 % de mediciones repetidas. La concurrencia es alta, con muchos relojes sincronizando a la vez.
Qué la haría rápida	La clave primaria ordenada por reloj y tiempo, la inserción de todo el lote en una sola instrucción y la partición temporal de la serie para descartar datos antiguos sin borrar fila por fila.

T3. Recomendación del día para un usuario (OLTP, lee por clave)

```
ids := GET recomendacion:{usuario}:{hoy};        -- almacen clave-valor, una lectura
SELECT recipe_id, nombre, calorias, proteina, tiempo_total
  FROM receta
 WHERE recipe_id IN (ids);                       -- hasta 10 filas por clave primaria
```

Propósito y usuario	Mostrar las recetas sugeridas para hoy. La ejecuta la app cuando el usuario abre la pantalla de inicio.
Clasificación	OLTP.
Justificación	Lee una clave y hasta diez recetas. Se ejecuta al menos una vez por usuario al día, con máximos antes de las comidas (el 91 % de las comidas se registra entre las 7 y las 9, las 12 y las 14 y las 19 y las 21, hora local). La latencia esperada es menor a 100 ms y la concurrencia, alta.
Qué la haría rápida	El acceso por clave en el almacén clave-valor, la clave primaria de `receta` y una caché para las recetas más sugeridas.

A1. Recetas candidatas según el gasto reciente de cada usuario (OLAP, responde el objetivo)

```
WITH gasto_reciente AS (
  SELECT id_usuario, AVG(kcal_dia) AS gasto_7d
    FROM (SELECT id_usuario, fecha_local, SUM(kcal_gastadas) AS kcal_dia
            FROM hechos_actividad_diaria
           WHERE fecha_local BETWEEN :hoy - 7 AND :hoy - 1
           GROUP BY id_usuario, fecha_local)
   GROUP BY id_usuario
)
SELECT g.id_usuario, r.id_receta, r.nombre, r.kcal_porcion, r.proteina_porcion
  FROM gasto_reciente g
  JOIN dim_receta r
    ON r.kcal_porcion BETWEEN :fraccion_min * g.gasto_7d
                          AND :fraccion_max * g.gasto_7d
 WHERE r.calificacion_promedio >= 4
   AND r.n_resenas >= 5
QUALIFY ROW_NUMBER() OVER (PARTITION BY g.id_usuario
                           ORDER BY r.calificacion_promedio DESC, r.n_resenas DESC) <= 10;
```

Propósito y usuario	Generar para cada usuario las diez recetas mejor calificadas cuyo aporte por porción cae en el rango que corresponde a su gasto. La ejecuta el proceso diario. En la E3 el promedio de siete días se reemplaza por la predicción del modelo.
Clasificación	OLAP.
Justificación	Solo lee. Recorre siete días de la tabla de hechos (unas 2.100 filas hoy, 1,9 millones con 271.907 usuarios) y las 522.517 recetas del catálogo, de las que 65.438 cumplen el filtro de calificación. Se ejecuta una vez al día, puede tardar minutos y no tiene concurrencia.
Qué la haría rápida	El almacenamiento por columnas, porque usa 3 columnas de los hechos y 6 de las 28 de la receta; la poda de particiones por fecha, y un rango calórico precalculado en `dim_receta` que convierte la comparación por rango en una igualdad.

A2. Balance diario entre gasto y comidas registradas (OLAP)

```
WITH gasto AS (
  SELECT id_usuario, fecha_local,
         SUM(kcal_gastadas) AS kcal_gastadas, SUM(pasos) AS pasos,
         SUM(minutos_sesion) AS minutos_sesion
    FROM hechos_actividad_diaria
   WHERE fecha_local BETWEEN :desde AND :hasta
   GROUP BY id_usuario, fecha_local
),
comidas AS (
  SELECT id_usuario, fecha_local, COUNT(*) AS comidas,
         SUM(kcal_porcion) AS kcal_registradas, SUM(proteina_porcion) AS proteina_registrada
    FROM hechos_comida
   WHERE fecha_local BETWEEN :desde AND :hasta
   GROUP BY id_usuario, fecha_local
)
SELECT g.id_usuario, g.fecha_local, f.dia_semana, g.kcal_gastadas,
       AVG(g.kcal_gastadas) OVER (PARTITION BY g.id_usuario ORDER BY g.fecha_local
                                  ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS gasto_media_7d,
       COALESCE(c.comidas, 0) AS comidas, c.kcal_registradas,
       g.kcal_gastadas - c.kcal_registradas AS balance_kcal
  FROM gasto g
  JOIN dim_fecha f ON f.fecha = g.fecha_local
  LEFT JOIN comidas c ON c.id_usuario = g.id_usuario AND c.fecha_local = g.fecha_local
 ORDER BY g.id_usuario, g.fecha_local;
```

Propósito y usuario	Comparar por usuario y día lo gastado con lo registrado como comida, junto con la media móvil del gasto. La usa el analista de producto a través del panel.
Clasificación	OLAP.
Justificación	Solo lee. Recorre todas las filas de ambas tablas de hechos dentro del rango (16.330 y 22.106 en los 61 días del extracto), agrupa y calcula una ventana. Se actualiza una vez al día y admite segundos de espera, con pocos analistas a la vez.
Qué la haría rápida	El almacenamiento por columnas, la partición por mes y, para el panel, un resumen precalculado por usuario y día que evite repetir el cruce en cada visita.

A3. Completitud y atraso de las mediciones por día y modelo (OLAP)

```
SELECT fecha_medicion, modelo, COUNT(*) AS mediciones,
       AVG(CASE WHEN ts_recepcion <= fecha_medicion + INTERVAL '30 hours'
                THEN 1 ELSE 0 END) AS fraccion_disponible_06h,   -- 06:00 UTC del dia siguiente
       QUANTILE(minutos_atraso, 0.5) AS atraso_mediano,
       QUANTILE(minutos_atraso, 0.9) AS atraso_p90
  FROM mediciones_limpias
 WHERE fecha_medicion BETWEEN :desde AND :hasta
 GROUP BY fecha_medicion, modelo
 ORDER BY fecha_medicion, modelo;
```

Propósito y usuario	Saber qué fracción de las mediciones de cada día ya estaba disponible a una hora de corte y cuánto se atrasa cada modelo. La usa el ingeniero de datos para fijar la hora del proceso diario y la ventana de reproceso.
Clasificación	OLAP.
Justificación	Solo lee. Es la consulta más pesada, porque recorre las mediciones al detalle: 15,2 millones de filas únicas en el extracto y cerca de 230 millones por día en el escenario con todos los usuarios. Se ejecuta de forma semanal o a demanda, puede tardar de segundos a minutos y no tiene concurrencia.
Qué la haría rápida	El almacenamiento por columnas con compresión, ya que usa 4 columnas, y la poda de particiones por fecha de medición cuando el rango es acotado. Es la que pienso usar para la medición de la E2.

F1. Resumen de hoy que el usuario ve al abrir la app (frontera)

```
SELECT SUM(pasos) AS pasos_hoy, SUM(calorias) AS kcal_hoy, MAX(ts_medicion) AS ultima_medicion
  FROM medicion
 WHERE device_id = :reloj
   AND ts_medicion >= :inicio_del_dia_local;
-- En la practica la app no ejecuta esta suma: lee el valor ya calculado.
valor := GET resumen:{usuario}:{hoy};
```

Propósito y usuario	Mostrar los pasos y las calorías acumuladas en el día. La ejecuta la app cada vez que el usuario abre la pantalla de inicio.
Clasificación	De frontera. Tiene forma analítica, porque agrega hasta unas 2.100 filas, pero uso operacional, porque es por una sola clave, muy frecuente y con respuesta inmediata. La trato como carga operacional.
Justificación	Solo lee. Se ejecuta varias veces al día por usuario, con latencia esperada menor a 100 ms y alta concurrencia. El dato tiene la frescura de la última sincronización, no la del instante.
Dónde se ejecuta	En el sistema operacional y sobre un valor precalculado: al recibir un lote (T2) se recalcula el resumen del día afectado y la app lo lee por clave. No se ejecuta en el Lakehouse, que va con horas de desfase, ni como suma sobre la serie en cada apertura, que competiría con la escritura de lotes. Si se pidiera para todos los usuarios a la vez, como en un ranking, sería una consulta analítica y se llevaría a una réplica de lectura o al Lakehouse.
Qué la haría rápida	El resumen precalculado en el almacén clave-valor y, como respaldo, la clave ordenada por reloj y tiempo de la serie.